P1: CPU Frequency vs. Memory Technology for a Prime Sieve in gem5

A prime sieve found all 78,498 primes up to 1,000,000 in every run. I tested two CPU, models TimingSimpleCPU and MinorCPU, at clock speeds from 1 to 3 GHz with three memory types (DDR3_1600, DDR3_2133 and LPDDR2). The sieve's 1 MB array is too big for the 256 kB L2 cache, so it has to use main memory.

Which CPU model is more sensitive to CPU frequency?

TimingSimpleCPU. Going from 1 to 3 GHz made it 1.84× faster, but MinorCPU only 1.68× faster. A faster clock speeds up the CPU's own work but not the time spent waiting for memory. TimingSimpleCPU runs one instruction at a time, so more of its time is CPU work that the clock speeds up. MinorCPU is pipelined, so it already does that work quickly, and more of its time is spent waiting for memory.

Which CPU model is more sensitive to the memory technology?

MinorCPU. The slower LPDDR2 memory made MinorCPU 23–41% slower, but TimingSimpleCPU only 15–31% slower. Both models lose about the same time waiting for memory, but MinorCPU's total runtime is shorter, so that waiting is a bigger share of it.

Is the sieve more sensitive to CPU frequency or memory technology?

CPU frequency. Changing the clock changed runtime by up to 1.84×, while changing the memory changed it by up to 1.41×. Most of the sieve's time is simple loop work inside the CPU. Memory matters more at higher clock speeds, because the CPU work gets faster while the time spent waiting for memory stays the same. The faster DDR3_2133 memory helped by only 2–5%.

Would the conclusions change with a different application?

Yes. A program that fits in the cache, such as a smaller sieve, would speed up close to 3× with the clock and barely care about the memory type. A program that constantly reads from main memory, such as one that follows pointers through a large data set, would gain little from a faster clock and depend mostly on memory speed.